

Outline:

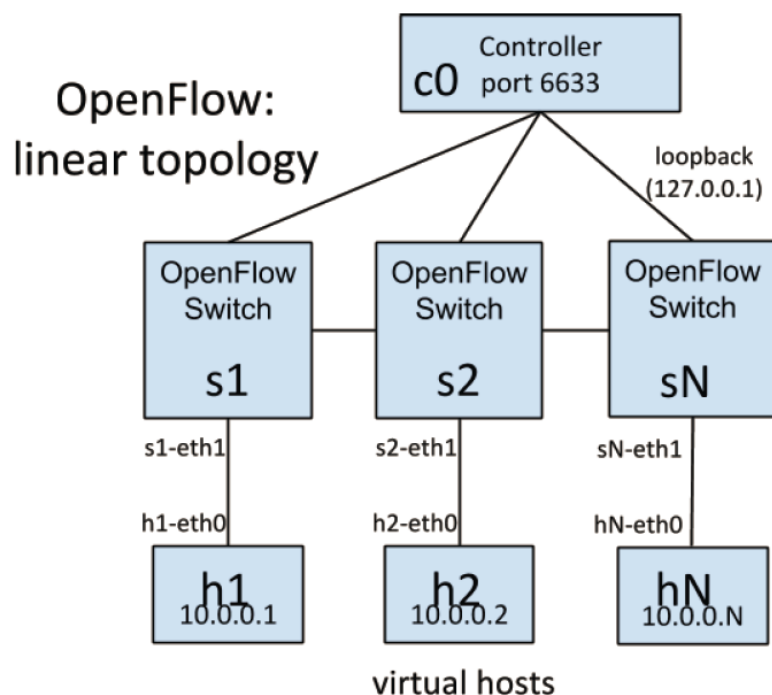
- Building complex topologies in Mininet,
- Create firewall application using POX controller (I2 and I3 based),
- Create switch application using POX controller.

Study environment:

- lab materials are uploaded on Moodle;
- All you need is to have machine with Linux (preferable ubuntu) with installed mininet (<https://github.com/mininet/mininet>) and Wireshark. Install also POX controller by cloning the repo : <https://github.com/noxrepo/pox> (all dependencies are installed on Ubuntu VM)
- Make checkout inside pox project *git checkout halosaur*
- Please attach two files *CustomTopology.py* and *fit-device.py* to moodle. Stubs for files can be found on moodle.
- tip: before exercises clean Mininet environment using *sudo mn --clean*

Building complex topologies in Mininet

So far you have used the command-line version of Mininet mostly for constructing simple topologies. Because python is the base technology behind the virtualization environment is you can also create your network topology in more programmatic ways. For example, linear topology (presented in the chart below) can be defined using the script.



Picture 1 Linear topology

Here is the script:

```
#!/usr/bin/python

from mininet.topo import Topo
from mininet.net import Mininet
from mininet.util import irange, dumpNodeConnections
from mininet.log import setLogLevel

class LinearTopo(Topo):
    "Linear topology of k switches, with one host per switch."

    def __init__(self, k=2, **opts):
        """Init.
        k: number of switches (and hosts)
        hconf: host configuration options
        lconf: link configuration options"""

        super(LinearTopo, self).__init__(**opts)

        self.k = k

        lastSwitch = None
        for i in irange(1, k):
            host = self.addHost('h%s' % i)
            switch = self.addSwitch('s%s' % i)
            self.addLink( host, switch)
            if lastSwitch:
                self.addLink( switch, lastSwitch)
            lastSwitch = switch

    def simpleTest():
        "Create and test a simple network"
        topo = LinearTopo(k=4)
        net = Mininet(topo)
        net.start()
        print "Dumping host connections"
        dumpNodeConnections(net.hosts)
        print "Testing network connectivity"
        net.pingAll()
        net.stop()

if __name__ == '__main__':
    # Tell mininet to print useful information
    setLogLevel('info')
    simpleTest()
```

Picture 2 Script for linear topology

The most important parts of API:

- *Topo*: basic class for the topology
- *addSwitch()*: adds switch to topology with device name as paramter,
- *addHost()*: adds host to topology with device name as paramter,
- *addLink()*: adds link between the elements (by default it is two-way communication, it can be customized)
- *Mininet*: main class for controlling and management of the network
- *start()*: method starts the emulated network

- `pingAll()`: global connectivity tests, ICMP Echo packets are sent between all hosts
- `stop()`: stops the network
- `net.hosts`: reference for all hosts in the network,
- `dumpNodeConnections()`: dump connections for set of nodes.

Environment also gives functionality for resource isolation and limitation, (classes *CPULimitedHost* and *TCLink*). The parameters can be set during environment initiation (e.g. `net = Mininet(topo=topo, host=CPULimitedHost, link=TCLink)`). Following operations can be run:

- `self.addHost(name, cpu=f)`: allows limit CPU usage for virtual hosts,
- `self.addLink(node1, node2, bw=10, delay='5ms', max_queue_size=1000, loss=1, use_htb=True)`: creates two-way connections between nodes.

Use *python* dictionaries to pass references for the link options.

```
linkopts = dict(bw=10, delay='5ms', loss=1, max_queue_size=1000,
use_htb=True)
'''
alternately: linkopts = {'bw':10, 'delay':'5ms', 'loss':1,
'max_queue_size':1000, 'use_htb':True}
'''

self.addLink(node1, node2, **linkopts)
```

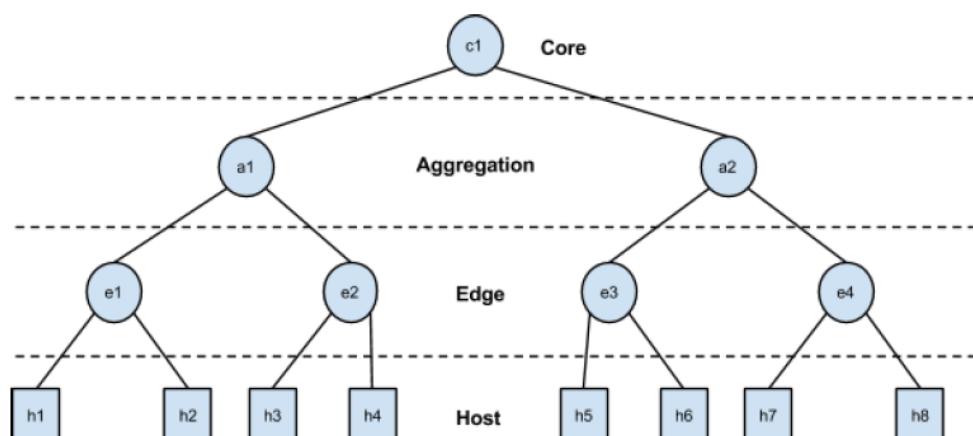
Picture 3 Dictionary as an argument for methods

To execute the script you need to use privilege mode (e.g. `sudo python LinearTopo.py`).

Task 1 – Build tree topology in Mininet

So as your first task try to execute:

- Implement the following topology with different parameters between the links (see above listing)



Picture 4 Topology

- download script *CustomTopology.py* from Moodle and based on it prepare your solution,
- in the source code, apply different options for the link QoS parameters,
- use *fanout* parameter in constructing the tree (amount of children),

- create all required hosts and switches, and build required topology
- add differenc link parameters to topology (all data are in the scripts)
- execute your script and verify its correctness using *ping* command - access time to different hosts should vary (e.g. *h1 ping h2* vs *h1 ping h8*)

TASK 1 - send the mininet file with CustomTopology.py topology defined

Create SDN application

POX controller is the default controller inside Mininet. It is simple controller written in Python, mainly used for prototyping and writing simple applications for network control. As an example application, we can mention hub, to see it open *hub.py* from folder *pox/forwarding*.

```
from pox.core import core
import pox.openflow.libopenflow_01 as of
from pox.lib.util import dpidToStr

log = core.getLogger()

def _handle_ConnectionUp (event):
    msg = of.ofp_flow_mod()
    msg.actions.append(of.ofp_action_output(port = of.OFPP_FLOOD))
    event.connection.send(msg)
    log.info("Hubifying %s", dpidToStr(event.dpid))

def launch ():
    core.openflow.addListenerByName("ConnectionUp", _handle_ConnectionUp)

    log.info("Hub running.")
```

Picture 5 Hub module

The most important elements of the script:

- When connection is up method *_handle_ConnectionUp()* is called,
- *connection.send(...)* method for sending *OpenFlow* messages to switch. *ofp_action_output* class holding actions (use by *ofp_packet_out* or *ofp_flow_mod*). Specify port on which you want to send the packet,
- *ofp_match* - class for specifying matching rules for messages, not specified field are not taken into account. Some of the most important fields:
 - *dl_src* - MAC source
 - *dl_dst* - MAC destination
 - *in_port* - ingress port of switch
- *ofp_flow_mod* - installs new flow, it defines matching rules, and actions as well as all parameters describing the message.

example:

```

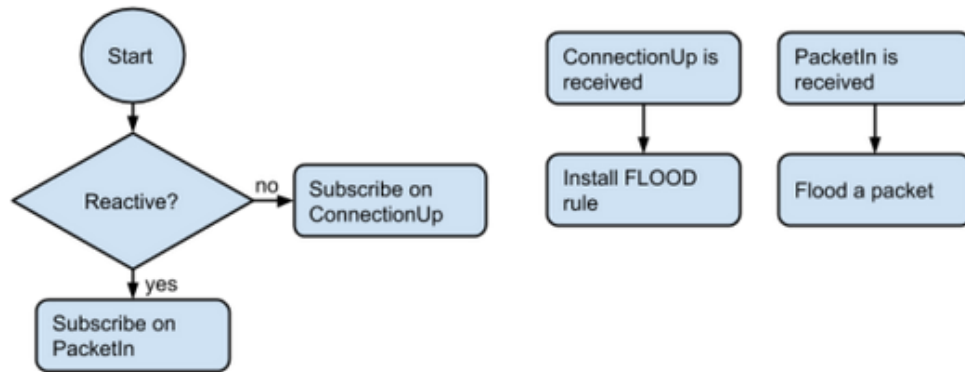
fm = of.ofp_flow_mod()

fm.match.in_port = 3

fm.actions.append(of.ofp_action_output(port = 4))

```

In hub example flows are created in proactive way.



Picture 6 Installing flows

Flows can be also added in reactive way - when new packet in message arrives new flows are installed (e.g. switch application)

Task 2 FITDevice application

In order to finish task perform following steps:

- modify your custom topology file by adding to options: when creating *Mininet* object pass option *autoSetMacs=True*, all MAC addresses will be assigned in linear order - the same as in the case of IP addresses (they are random with the option) and *autoStaticArp=True* automatically sets the ARP addresses on hosts,
- next create new topology based on *CustomTopology.py* use external controller (uncomment)

```

net = Mininet(topo=topo, link=TCLink, controller=RemoteController);

net.addController('c3', controller=RemoteController, ip='your_ip',
port=6633)

```

- **Implement L2 firewall, install policies in proactively**
- Do not forget to checkout ~/pox project to proper version: *git checkout halosaur*
- Copy file *fit-device.py* to the ~/pox/pox/misc location
- using *fit-device.py* and *firewall-policies.csv* implement your simple firewall, stub files should be present in the downloaded archive. E.g. following rule in file:

```
1,00:00:00:00:00:01,00:00:00:00:00:08
```

Blocks traffic from MAC 01 to MAC 08.

- your firewall installs rules proactively on all devices based on L2 addresses (once the *_connectionUp* handler is called), check implementation from *fit-device.py*

- start external POX controller with your application, the default implementation add support for hub

```
pox.py misc.fit-device
```

- based on the presented *hub.py* sample and the learning switch module *l2_learning* (pay attention to message construction), build your application, implement *firewall_l2* method
- set proper priorities when sending the message (*msg.priority*), firewall rules should be executed before the learning switch ones,
- load firewall database from CSV file, use *EthAddr* for address conversion

```
msg.match.dl_dst = EthAddr(row[2])
```

- verify correctness of your implementation using *ping* command, some packets should be blocked
- **Implement L3 firewall, install policies in proactively**
- Implement method *firewall_l3*, take Ip addresses from the second file
- **Implement learning switch, install policies reactively**
- Modify the *_handle_PacketIn* method and uncomment the switch method
- Based on description implement learning switch device (you may take a look on implementation from *l2_learning* module)

TASK 2 - send the pox implemenetation with your fit-device app

References

1. POX documentation, <https://noxrepo.github.io/pox-doc/html/>